MANIPAL UNIVERSITY JAIPUR

University u der Sectio 2(f) of the UGC Act

Centre for Distance & Online Education

Manipal University Jaipur

Internal Assignment Submission – April 2025

Programme: Bachelor of Computer Applications (BCA)

Semester: IInd

Course Code & Name: DCA1210 – Object-Oriented Programming Using C++

Assignment Sets: Set-1 & Set-2

Session: April 2025

Credits: 4

Submitted By:

Name: Hirdyansh Varshney

Roll Number: 2414500172

Email ID: 2414500172@mujonline.edu.in

Declaration:

I hereby declare that the content of this assignment is original and has been prepared by me.

It has not been copied from any source, and plagiarism has been strictly avoided.

Signature: Hirdyansh Varshney

Set–1

Q1.

Answer : Programming is the foundation of software development, and several methodologies existed with the passage of time. Procedural Programming (embraced by C) and Object-Oriented Programming (embraced by C++) are two most used methodologies. Both differ in the design philosophy, structure, and application.

1. Programming Paradigm:

C (Procedural Programming): Top-down by design. Program is segmented into procedures or functions.

C++ (Object-Oriented Programming): It is bottom-up in nature. The program is divided into classes and objects with functions and data as a whole.

2. Focus of Programming:

C: Interested in processes and procedures. Data is supplementary and typically dispersed across the world.

C++: Data-oriented and object-oriented. Functions have close linkage to data on which they operate.

3. Data Handling:

C: Information is everywhere accessible to all functions, particularly with global variables. It can lead to unintentional modification and makes debugging more difficult.

C++: Applies encapsulation to restrict access to data. Data can be accessed only via member functions of a class, thereby further increasing data security.

4. Code Reusability:

There is limited and manual reuse of code. Functions are reusable but data structures are not flexible.

C++: Inheritance is provided, enabling new classes to inherit and build upon existing class functionality.

5. Real-World Modeling:

It is hard to simulate real-world objects. Example: Modeling an automobile requires treatment of various variables and functions separately.

C++: Uses objects (such as Car, Engine, etc.) to represent real-life concepts, rendering the design structured and intuitive.

6. Function Overloading and Polymorphism:

Does not support function overloading and polymorphism.

C++: Polymorphism is supported, i.e., the function is utilized differently for different classes. Function and operator overloading are also supported.

7. Extensibility:

New data types or features bring significant code changes.

C++: Templates and classes allow you to easily introduce and modify code without disrupting previously implemented functionality.

8. Security and Maintenance: Programs become increasingly difficult to maintain as they expand because of global data and loose structure. C++: Offers modularity, abstraction, and encapsulation and therefore the code is more maintainable and secure. Conclusion: C is an excellent language for small, function-oriented programs, but when programs grow in size and complexity, its limitations are exposed. C++ remedies these with the power of OOP principles like encapsulation, inheritance, and polymorphism. This makes C++ a better language to use when developing large-scale applications that are maintainable and secure.

Q2.

Answer : In C++, functions are commonly used to divide a program into smaller, manageable parts. However, every time a function is called, there is some overhead in jumping to that function's location in memory. To reduce this overhead, C++ introduces inline functions, which are a powerful feature for increasing execution efficiency in certain situations.

- **Definition of Inline Function:**

An inline function is a function for which the compiler replaces the function call with the actual code of the function during compilation. This avoids the overhead of a function call and can improve performance, especially for small and frequently used functions.

To declare an inline function, we use the keyword inline before the function definition.

Example:

```
#include <iostream>

using namespace std;

inline int square(int x) {

   return x * x;

}

int main() {

   int num = 5;

cout << "Square of " << num << " is " << square(num);

   return 0;
```

}

In the above example, the call to square(num) will be replaced by num * num during compilation, avoiding the function call overhead.

- **Advantages of Inline Functions:**

1. Faster Execution:

- Inline functions reduce the overhead of function calls.
- Since the function code is directly inserted at the call location, it saves time used in jumping to function code and returning back.

2. Better for Small Functions:

- Especially useful for functions with 1–2 lines of logic like mathematical calculations, getters, etc.

3. Improves Code Readability:

- Helps keep the logic modular, but without the performance penalty of regular functions.

4. Useful in Templates:

- In function templates, inline functions improve generic programming performance by reducing function call overhead.

5. Compiler Optimization:

- It gives the compiler more freedom to optimize the final executable by replacing code inline.

Limitations / When Not to Use Inline:

- Should not be used for large functions — it increases the size of the binary file.
- Cannot be used with functions containing loops, switch statements, or static variables.
- Final decision to inline is taken by the compiler, not the programmer.

- **Conclusion:**

Inline functions are a feature introduced in C++ to optimize small and frequently used functions. By embedding the function code directly into the program, they reduce function call overhead, leading to faster execution. However, developers must use them carefully to avoid large code size and inefficiency in larger programs.

Q3.

Answer : In programming, errors can occur at any stage—during input, file handling, or even simple calculations. These errors are called exceptions, and if not handled properly, they can

cause a program to crash. To manage such unexpected situations, C++ provides a powerful feature called *exception handling*.

Exception handling allows the programmer to detect, manage, and respond to runtime errors in a structured way, making the program more reliable and user-friendly.

- **What is Exception Handling?**

Exception handling in C++ is a mechanism that helps you detect and handle errors during program execution without abruptly terminating the program. When an error (exception) occurs, it is "thrown" and then "caught" using special keywords: try, throw, and catch.

- **Syntax of Exception Handling**

```
try {
   // Code that might cause an exception
   if (some_error)
      throw error_code;
}
catch (datatype var) {
   // Code to handle the error
}
```

Explanation of Keywords:

try block:

- Used to wrap the code that may cause an error.
- If an exception occurs inside the try block, control moves to the matching catch block.
- Example:

```
try {
   int a = 10, b = 0;
   if (b == 0)
      throw "Division by zero error!";
}
```

throw keyword:

- Used to throw an exception manually.
- It can throw any data type: integer, float, char, string, object, etc.
- When a throw is executed, the program immediately looks for a matching catch.

catch block:

- Used to catch and handle the exception.
- It must be placed immediately after the try block.
- Example:

```
catch (const char* msg) {
    cout << "Error: " << msg << endl;
}
```

Example Program:

```
#include <iostream>
using namespace std;
int main() {
    int a = 10, b = 0;
    try {
        if (b == 0)
            throw "Cannot divide by zero!";
        cout << "Result: " << a / b;
    }
    catch (const char* msg) {
        cout << "Exception occurred: " << msg << endl;
    }
    return 0;
}
```

Output:

Exception occurred: Cannot divide by zero!

Advantages of Exception Handling:

1. Improves program stability – avoids crashes.
2. Separates error-handling code from regular code.
3. Helps in debugging and user-friendly error messages.
4. Supports multiple catch blocks for handling different types of errors.

- **Conclusion:**

Exception handling is a crucial part of robust C++ programming. It allows developers to handle unexpected situations using the try, throw, and catch mechanism in a clean and

organized way. By using these tools, programmers can ensure that their software behaves reliably even when things go wrong.

Set–2

Q4.

Answer: In C++, a pointer is a special type of variable that stores the memory address of another variable. Instead of holding data directly, it "points" to the location in memory where the data is stored. This is a powerful concept in C++ that allows efficient memory access and manipulation.

Understanding pointers is essential because they are widely used in dynamic memory allocation, arrays, functions, and even object-oriented programming.

- **Declaration and Syntax**

To declare a pointer, use the * symbol:

int *ptr; // declares a pointer to an integer

Here, ptr is a pointer that can store the address of an int type variable.

Example of Basic Pointer Use:

```
#include <iostream>

using namespace std;

int main() {
    int num = 10;
    int *ptr = &num; // storing address of num
    cout << "Value of num: " << num << endl;
    cout << "Address of num: " << &num << endl;
    cout << "Value in pointer ptr: " << ptr << endl;
    cout << "Value pointed by ptr: " << *ptr << endl;
    return 0;
}
```

Output:

Value of num: 10

Address of num: 0x61ff08

Value in pointer ptr: 0x61ff08

Value pointed by ptr: 10

- Key Terms Related to Pointers:

Term	Description
& (Address-of)	Gets the address of a variable.
(Dereference)	Accesses the value stored at an address.

Why Pointers Are Useful

1. Dynamic Memory Allocation:

Pointers allow creation of variables and arrays at runtime using operators like new and delete.

```
int *p = new int;
*p = 50;
delete p;
```

2. Function Arguments (Call by Reference):

Pointers can be used to pass values by reference, so changes inside the function reflect outside too.

```
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

3. Arrays and Strings:

Pointers make it easier to manipulate arrays and strings, especially in loops.

4. Objects and Classes:

You can also create pointers to objects, and use them to access class members using -> operator.

Example: Swapping Values Using Pointers

```
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
int main() {
    int a = 5, b = 10;
```

```
    swap(&a, &b);
    cout << "a = " << a << ", b = " << b;
}
```

- **Conclusion:**

Pointers are one of the most powerful features of C++, giving the programmer direct access to memory. They improve the efficiency of programs and are especially useful in situations like dynamic memory, handling arrays, and working with objects. However, they must be used carefully to avoid issues like memory leaks or segmentation faults.

Q5.

Answer: In C++, a constructor is a special member function of a class that is automatically called whenever an object of that class is created. Its main purpose is to initialize the data members of the class.

Unlike normal functions, a constructor:

- Has the same name as the class.
- Has no return type, not even void.
- Can be overloaded, i.e., there can be multiple constructors in a class with different parameter lists.

Types of Constructors:

1. Default Constructor – Takes no arguments.

```
class A {
    public:
    A() {
        cout << "Default Constructor Called";
    }
};
```

2. Parameterized Constructor – Takes arguments to initialize variables.

```
class A {
    int x;
    public:
    A(int a) {
        x = a;
```

```
    }
};
```

3. Copy Constructor – Creates a copy of an existing object.

```
A(A &obj) {
    x = obj.x;
}
```

Importance of Constructors:

- They ensure objects are properly initialized at the time of creation.
- Help in code readability and structure.
- Useful in object copying, overloading, and dynamic memory management.

Conclusion:

Constructors are a key feature of object-oriented programming in C++. They automate initialization and make object handling more structured and efficient.

Q6.

Answer : In C++, a destructor is a special member function that is automatically called when an object goes out of scope or is explicitly deleted. Its main purpose is to release resources or perform clean-up tasks such as memory deallocation.

A destructor:

- Has the same name as the class, but is preceded with a tilde ~.
- Takes no arguments and has no return type.
- Cannot be overloaded.

- Syntax of Destructor:

```
class MyClass {
public:
    ~MyClass() {
        // Cleanup code
        cout << "Destructor called";
    }
};
```

Whenever an object of MyClass is destroyed, the destructor will be automatically executed.

Why Destructors are Important:

1. Memory Management: Helps in freeing memory allocated using new.
2. Avoids Memory Leaks: Cleans up resources to prevent system overload.
3. Automatic Execution: No need to call it manually.

Example:

```
#include <iostream>
using namespace std;
class Test {
public:
    Test() {
        cout << "Constructor\n";
    }
    ~Test() {
        cout << "Destructor\n";
    }
};
int main() {
    Test obj;
    return 0;
}
```

Output:

nginx

Constructor Destructor

- **Conclusion:**

Destructors are an essential part of object lifecycle in C++. They help in proper resource cleanup and make programs more efficient and error-free.